RETO 3 · PRUEBA B · PSP · 2º DAM

UDITflix: El Optimizador de Descarga Masiva de Assets

Motor concurrente · Latencia aleatoria, prioridades y orquestación con ArrayList

Módulo	0490 · Programación de Servicios y Procesos (2º DAM)
RA vinculado	RA2 · Programación de aplicaciones multihilo: creación, arranque, sincronización con `join()`, estados con `getState()`, prioridades, hilos demonio y no-determinismo
Tecnología	Java (`Runnable`, `Thread`) en IntelliJ IDEA
Modalidad	Individual · sin herramientas de IA generativa
Entrega	Repositorio GitHub (código + `README.md`) + vídeo de defensa técnica (< 3 min)

1. Caso de estudio: la pantalla de carga que nunca termina

> **Informe de incidencia · Equipo de la app móvil (Flutter) de UDITflix**
> **Síntoma:** los usuarios tardan demasiado en ver la pantalla de login al abrir la app.
> **Causa raíz:** antes de permitir el login, la app necesita descargar de los servidores de caché varios módulos de datos y assets gráficos, y ahora lo hace **de uno en uno**.
> **Solicitud:** un motor de precarga que descargue los módulos **en paralelo**, que dé preferencia a los datos críticos frente a la analítica secundaria y que habilite el login solo cuando todo esté listo.

Eres el/la ingeniero/a de backend asignado/a. Simularás en Java la descarga concurrente de los módulos y analizarás cómo se comporta el planificador del sistema operativo cuando asignas prioridades distintas.

2. Punto de partida

No empiezas de cero. Parte de la **base v0** que construimos en clase: la clase `TareaTranscodificacion` (que implementa `Runnable`) y la clase `MainUDITflix`, que crea 4 hilos con nombre propio (`WORKER-1`...), los lanza con `start()`, espera con `join()` y calcula tiempos. En esta prueba mantienes las latencias aleatorias de la base, pero los hilos pasan a gestionarse en una lista y les asignas prioridades. Puedes adaptar la clase de tarea (por ejemplo, a `TareaDescarga`) en lugar de partir de cero.

> **Recuerda:** `run()` ejecuta el código en el hilo actual (sin concurrencia). Solo `start()` crea un hilo nuevo de verdad.

3. Requisitos técnicos

RT1 · Cuatro módulos descargados en paralelo

Módulo	Contenido	Tipo	Prioridad obligatoria
`catalogo-metadatos`	Catálogo, géneros, fichas	Crítico	`Thread.MAX_PRIORITY`
`sesion-y-perfil`	Token de sesión y perfil	Crítico	`Thread.MAX_PRIORITY`
`portadas-assets`	Pósters y banners	Estándar	`NORM_PRIORITY` (por defecto)
`analiticas`	Telemetría de uso	Secundario	`Thread.MIN_PRIORITY`

RT2 · Latencia aleatoria (como en la base)

Cada módulo simula su descarga con una **latencia aleatoria** obtenida con `Random`, igual que en la base de clase (`1000 + azar.nextInt(3001)` ms) y usando `Thread.sleep()`. Imprime la latencia que le ha tocado a cada módulo.

RT3 · Prioridades

Asigna las prioridades de la tabla con `setPriority()` **antes** de `start()` e imprime `getPriority()` al arrancar y al terminar cada hilo.

RT4 · Orquestación con una lista dinámica

Guarda los hilos en un `ArrayList<Thread>`. Lánzalos con un bucle y espéralos con **otro bucle distinto** que haga `join()`. Los bucles deben quedar limpios: añadir un quinto módulo debería exigir tocar **una sola línea**.

RT5 · La puerta del login

Solo cuando todos los hilos hayan terminado, `main` imprime «Login habilitado» junto con el **tiempo total real**, la **duración del módulo más lento** y la **suma de todas las latencias**, como hace tu base.

RT6 · Análisis breve del planificador (en el README)

Ejecuta el programa **al menos 3 veces**, anota el orden de finalización de los módulos y escribe un análisis breve (5-8 líneas): ¿terminaron antes los hilos de prioridad alta? ¿Por qué crees que sí o que no en tu sistema operativo?

> **Importante:** en Java las prioridades son una *sugerencia* al planificador. Su efecto depende del sistema operativo, de la JVM y del número de núcleos de tu equipo. Si no observas diferencia, **es un resultado válido**: lo evaluable es que lo midas, lo documentes y lo expliques.

Salida orientativa (los valores variarán en tu equipo)

```
[MAIN] Descargando 4 módulos en paralelo...
--> [DL-catalogo]   prio=10  latencia=1840 ms
--> [DL-sesion]     prio=10  latencia=1120 ms
--> [DL-portadas]   prio= 5  latencia=3610 ms
--> [DL-analitica]  prio= 1  latencia=2950 ms
<-- [DL-sesion]     LISTO
...
[MAIN] Todos los módulos listos  ->  LOGIN HABILITADO
Tiempo total real:        ≈ 3610 ms
Duración del más lento:     3610 ms
Suma de todas:              9520 ms
```

4. Ampliación (opcional)

- **Pantalla de carga demonio:** un hilo que imprima «Cargando...» cada 300 ms con `setDaemon(true)`. Explica en el README por qué el programa termina cuando acaban los módulos aunque ese hilo nunca acabe por sí mismo.
- **Experimento del planificador:** añade tras la latencia una fase de procesado que consuma CPU (un bucle de cálculo de 1-2 s) y repite el análisis. Un hilo dormido no compite por la CPU; uno que calcula, sí. ¿Cambia lo que observas?

5. Entregables

☐ **Repositorio de GitHub** con el proyecto completo, que compile y se ejecute en IntelliJ.

☐ `README.md` completado a partir de la plantilla, con el **análisis breve del planificador** y las preguntas de metacognición.

☐ **Vídeo de defensa técnica** (< 3 min) con el enlace incluido en el README.

6. Vídeo de defensa técnica (< 3 minutos)

Además del código, grabarás un vídeo corto defendiendo tu solución. No es una presentación: es una conversación técnica contigo mismo/a delante del código.

Qué debes mostrar en pantalla

- **IntelliJ con el código**: las clases del proyecto y, en concreto, el fragmento donde creas, lanzas y sincronizas los hilos.
- **El programa en ejecución** y la **consola con las trazas** (amplía la fuente para que se lea bien).
- **Al menos dos ejecuciones seguidas**, para enseñar qué cambia entre una y otra.
- **Las prioridades impresas** y el orden de finalización en dos ejecuciones.
- Brevemente, **tu análisis del planificador** del README.

Qué debes explicar (con tus palabras)

- **El problema**: qué le pasa a la empresa y qué pide el cliente.
- **Cómo lo resolviste**: la estructura de tu solución, paso a paso.
- **La justificación técnica de tus decisiones**: por qué esa estructura y no otra.
- **Qué diferencias observaste al cambiar las prioridades en tu sistema operativo** (o por qué no hubo ninguna).

Guion orientativo

Tramo	Contenido
0:00 – 0:25	El problema y el objetivo, en dos frases.
0:25 – 1:15	Recorrido por el código: dónde nacen, arrancan y se esperan los hilos.
1:15 – 2:15	Ejecución en vivo y lectura de la consola. Segunda ejecución y comparación.
2:15 – 2:50	Decisiones técnicas y el problema que más te costó depurar.
2:50 – 3:00	Cierre: qué has aprendido.

Normas: tu propia voz, sin leer el código línea a línea y sin IA. Lo que supere los 3 minutos no se evaluará. No subas el vídeo al repositorio: súbelo a la plataforma que indique el docente y pega el enlace en el `README.md`.

7. Criterios de aceptación

Criterio (RA2)	Evidencia esperada
Creación y arranque	4 hilos concurrentes con nombre propio, lanzados con start().
Sincronización	Los hilos se guardan en un ArrayList y se esperan con join() en un bucle; el login solo se habilita al final.
Latencia y no-determinismo	Latencias aleatorias con Random (Thread.sleep); el orden de llegada cambia entre ejecuciones y se explica. El total coincide con el módulo más lento, no con la suma.
Prioridades	MAX_PRIORITY para críticos y MIN_PRIORITY para analítica, verificables con getPriority() en la traza.
Análisis del planificador	Análisis breve en el README con al menos 3 ejecuciones y conclusión razonada (también si no hay diferencia).
Documentación y vídeo	README completo con las preguntas de metacognición respondidas; vídeo de menos de 3 min.

8. Declaración de autoría

Este reto **no permite herramientas de IA generativa**. Puedes consultar únicamente los apuntes de clase, la documentación oficial de Java y la de IntelliJ IDEA. Al entregar confirmas que has diseñado, programado y depurado tu solución y que puedes explicarla línea a línea.